

Chapter 23

Agent Skills

As agents evolve from monolithic prompt-and-tool systems into modular architectures, a key design question emerges: *how should an agent’s capabilities be organized, discovered, and composed?* The answer increasingly converges on the concept of **skills** — discrete, reusable units of behaviour that can be loaded, combined, and swapped without retraining.

The idea was popularized by Voyager [362], which demonstrated that an LLM agent in Minecraft could accumulate a growing library of executable code skills, each verified and stored for later reuse. The same principle applies to production agents: skills encapsulate domain expertise in a composable, versionable format that scales beyond what any single prompt can hold. Skills frequently wrap MCP servers (Chapter 22) for tool access, connecting the skill abstraction to the standardized tool layer.

23.1 What Is a Skill?

A **skill** is a self-contained capability module that gives an agent expertise in a specific domain or task. Unlike a raw tool (which exposes a single function), a skill encompasses:

- **System prompt augmentation:** Domain-specific instructions, constraints, and persona elements injected into the agent’s context.
- **Tool bindings:** One or more tools the skill requires (APIs, MCP servers, local commands).
- **Knowledge:** Reference material, examples, or few-shot demonstrations the agent needs to execute the skill correctly.
- **Workflow logic:** Multi-step procedures, decision trees, or conditional flows that guide the agent through complex tasks.
- **Guardrails:** Skill-specific safety constraints, output format requirements, and validation rules.

Skill vs. Tool vs. Agent		
Concept	Scope	Example
Tool	Single function call	<code>web_search(query)</code>
Skill	Coherent capability (prompts + tools + knowledge)	“Research Analyst” skill
Agent	Autonomous entity with multiple skills	A coding assistant

A tool is a hammer. A skill is knowing *how to frame a house*. An agent is the carpenter who selects which skills to apply.

23.2 Skill Architecture Patterns

23.2.1 Static Skill Loading

The simplest pattern: skills are loaded at agent initialization based on configuration. The agent always has access to all its skills.

```
# Pseudocode -- framework-agnostic pattern
agent = Agent(
    model="claude-sonnet-4-20250514",
    skills=["code-review", "documentation", "testing"],
    # Each skill adds prompts, tools, and knowledge to the agent
)
```

Pros: Simple, predictable, low latency.

Cons: Context window waste when skills are unused; doesn’t scale to hundreds of skills.

23.2.2 Dynamic Skill Discovery

The agent selects which skills to activate based on the current task. A skill router (often a lightweight classifier or embedding-based matcher) determines relevance:

```
# Pseudocode -- framework-agnostic pattern
relevant_skills = skill_router.match(
    user_request=message,
    available_skills=skill_registry,
    max_skills=3
)
agent.activate(relevant_skills)
```

Pros: Scales to large skill libraries; context-efficient.

Cons: Routing errors can miss relevant skills; adds latency.

23.2.3 Hierarchical Skill Composition

Skills can depend on other skills, forming a DAG. A high-level skill (e.g., “Deploy Application”) may invoke sub-skills (“Run Tests”, “Build Docker Image”, “Update DNS”):

- Skills declare dependencies explicitly
- The orchestrator resolves the dependency graph before execution
- Sub-skills can be shared across multiple parent skills

23.3 Case Study: Anthropic’s Agent Design

Anthropic’s approach to agent architecture [10] provides one of the clearest articulations of skill-based agent design in production. Their philosophy emphasizes **simplicity over complexity** and **composable building blocks over monolithic frameworks**. (These patterns are also covered from an orchestration perspective in Chapter 20.)

23.3.1 Core Principles

1. **Start with the simplest solution.** Don’t reach for agentic patterns until simpler approaches (single LLM call, retrieval + generation) have been tried and found insufficient.
2. **Workflows vs. agents.** Anthropic distinguishes between:

- **Workflows:** Predefined orchestration of LLM calls — deterministic control flow with LLM steps at specific nodes. More predictable, easier to debug.
- **Agents:** The LLM dynamically decides what to do next — tool selection, iteration count, and stopping criteria are all model-driven. More flexible, harder to control.

3. **Augmented LLM as the atomic unit.** The primitive is never a bare model—it is always a model bundled with its retrieval sources, callable tools, and persistent memory. This composite unit is, in practice, a skill-equipped model.

23.3.2 Building Block Patterns

Anthropic identifies five composable workflow patterns that function as skill templates:

Table 23.1: Anthropic’s composable agent patterns.

Pattern	Mechanism	When to Use
Prompt Chaining	Sequential LLM calls where each step’s output feeds the next. Gates between steps validate intermediate results.	Multi-step transformations with clear decomposition
Routing	A classifier or LLM directs input to a specialized handler (skill) based on task type.	Distinct task categories requiring different expertise
Parallelization	Multiple LLM calls run simultaneously — either sectioning (split task) or voting (same task, aggregate).	Independent subtasks; or confidence via consensus
Orchestrator–Workers	A central LLM breaks the task into subtasks, delegates to worker LLMs, then synthesizes results.	Complex tasks where subtasks aren’t predictable in advance
Evaluator–Optimizer	One LLM generates, another evaluates; iterate until quality threshold is met.	Tasks with clear quality criteria (code, writing)

23.3.3 The Augmented LLM

In Anthropic’s framing, the fundamental unit is not the bare model but the **augmented LLM**:

$$\text{Augmented LLM} = \text{Model} + \text{Retrieval} + \text{Tools} + \text{Memory}$$

This maps directly to the skill concept: each skill configures which retrieval sources, tools, and memory stores the model has access to for a specific task. The skill boundary defines what the model *can see and do* within a particular invocation.

23.3.4 Practical Implications

Anthropic’s Key Insight

The most effective agents aren’t the most complex ones. They are **simple loops with good tools**:

```
while not done:
    action = llm.decide(context, tools)
    result = execute(action)
    context.append(result)
    done = llm.should_stop(context)
```

The intelligence comes from (1) the model's capability, (2) the quality of tool descriptions, and (3) the clarity of the task framing — not from elaborate orchestration logic. Skills provide the structure for (2) and (3).

Design recommendations from Anthropic's approach:

- **Keep agent loops simple:** Avoid over-engineering the control flow. Let the model decide.
- **Invest in tool quality:** Detailed, unambiguous tool descriptions are more valuable than complex routing logic.
- **Use structured outputs:** Force the model to output decisions in parseable formats (JSON, function calls) — reduces skill execution errors.
- **Build in recovery:** Skills should handle errors gracefully — retry with different parameters, ask for clarification, or escalate to a human.
- **Limit scope per skill:** A skill that tries to do everything will do nothing well. Narrow, well-defined skills compose better than broad ones.

23.4 Skill Lifecycle

1. **Discovery:** The system identifies which skills are available (registry, marketplace, local definitions).
2. **Selection:** Based on the user request, relevant skills are matched and loaded.
3. **Activation:** Skill prompts, tools, and knowledge are injected into the agent's context.
4. **Execution:** The agent uses the skill's capabilities to accomplish the task.
5. **Deactivation:** Skill context is removed to free context window space for subsequent tasks.
6. **Learning:** Execution results may update the skill's few-shot examples or fine-tune routing.

23.5 Skill Registries and Marketplaces

Production skill systems require infrastructure:

- **Skill manifest:** A structured description (name, capabilities, required tools, input/output schema) enabling automatic discovery and routing.
- **Version control:** Skills evolve; agents need to pin specific versions for reproducibility.
- **Dependency resolution:** Skills may require specific MCP servers, API keys, or other skills.
- **Permission model:** Not all agents should have access to all skills (security, cost, capability boundaries).
- **Marketplace:** Organizations can publish, share, and install skills — analogous to package managers for code.

Skill Manifest Example

A skill manifest declares everything an orchestrator needs to load and invoke a skill. No industry-standard schema exists yet; below is an illustrative format that captures common fields across real implementations (Anthropic MCP, OpenAI function specs, LangChain tool definitions):

```
// Illustrative schema -- not a specific SDK format
```

```
{
  "name": "code-review",
  "description": "Review code changes for bugs, style, and security issues",
  "version": "2.1.0",
  "requires": {
    "tools": ["file_read", "grep", "git_diff"],
    "mcp_servers": ["github"],
    "models": ["claude-sonnet-4-20250514"]
  },
  "input_schema": {
    "type": "object",
    "properties": {
      "repo": {"type": "string"},
      "pr_number": {"type": "integer"}
    }
  },
  "prompts": ["skills/code-review/system.md"],
  "knowledge": ["skills/code-review/style-guide.md"]
}
```

23.6 Skills vs. Fine-Tuning

A natural question: why use runtime skill injection instead of fine-tuning the model?

Table 23.2: Skills (in-context) vs. fine-tuning for adding capabilities.

Dimension	Skills (In-Context)	Fine-Tuning
Deployment speed	Instant	Hours–days
Flexibility	Swap/combine at runtime	Fixed at training time
Context cost	Uses context window	Zero runtime cost
Deep behavior change	Limited by context length	Deep parametric change
Multi-tenant	Different skills per user	Same model for all
Maintenance	Update text files	Retrain on new data

In practice, the two approaches are complementary: fine-tuning provides *base capabilities* (instruction following, tool use format, reasoning), while skills provide *task-specific expertise* layered on top at runtime.